

Lesson 6 — Completing and Testing Sparse Matrix Addition in C

1. Where We Are Now

You have already learned the main ideas needed for sparse matrix addition.

Lesson	Main idea
Lesson 1	A sparse matrix stores mostly zero values, so we store only non-zero values.
Lesson 2	Coordinate List representation stores each non-zero value as row, column, and value.
Lesson 3	A sparse matrix can be displayed as a full matrix using nested loops and a tracker variable.
Lesson 4	Sparse matrix addition uses a merge-style algorithm with <code>i</code> , <code>j</code> , and <code>k</code> .
Lesson 5	The <code>add()</code> function creates a result matrix, merges two sparse lists, and returns the result pointer.
Lesson 6	We complete the full sparse matrix addition program in C using <code>malloc</code> , <code>free</code> , and test cases.

Lesson 6 is mainly about completing and testing the C program.

The addition algorithm itself is the same as Lesson 5. The big change is that we now use C-style dynamic memory allocation with `malloc` instead of C++ `new`.

2. Main Goal of Lesson 6

The goal of Lesson 6 is to write a complete C program that can:

1. Create the first sparse matrix.
2. Create the second sparse matrix.
3. Add the two sparse matrices.
4. Display the first matrix.
5. Display the second matrix.
6. Display the sum matrix.
7. Release dynamically allocated memory using `free()`.

The result matrix is created inside the `add()` function and returned to `main()` as a pointer.

3. Key Difference Between Lesson 5 and Lesson 6

Lesson 5 used C++-style allocation:

```
sum = new Sparse;  
sum->ele = new Element[S1->num + S2->num];
```

Lesson 6 uses C-style allocation:

```
sum = (struct Sparse *) malloc(sizeof(struct Sparse));  
sum->ele = (struct Element *) malloc((s1->num + s2->num) * sizeof(struct  
Element));
```

So the idea is the same, but the memory allocation method changes.

C++ style	C style
<code>new Sparse</code>	<code>malloc(sizeof(struct Sparse))</code>
<code>new Element[size]</code>	<code>malloc(size * sizeof(struct Element))</code>
<code>delete</code> or <code>delete[]</code>	<code>free()</code>
<code>nullptr</code>	<code>NULL</code>

4. Important Vocabulary

Term	Simple meaning
Sparse matrix	A matrix where most values are zero.
Non-zero element	A matrix value that is not zero.
Coordinate List	Sparse representation that stores row, column, and value.
<code>m</code>	Number of rows in the matrix.
<code>n</code>	Number of columns in the matrix.
<code>num</code>	Number of non-zero elements.
<code>ele</code>	Pointer to the dynamic array of non-zero elements.
<code>i</code>	Pointer/index for the first sparse matrix.

Term	Simple meaning
<code>j</code>	Pointer/index for the second sparse matrix.
<code>k</code>	Pointer/index for the result sparse matrix.
<code>malloc</code>	Creates dynamic memory in C.
<code>free</code>	Releases memory created using <code>malloc</code> .
<code>NULL</code>	Means no valid pointer/result.

5. Data Structures Used

The program uses two structures:

1. `Element`
2. `Sparse`

6. The `Element` Structure

One `Element` stores one non-zero matrix value.

```
struct Element {
    int i; // row index
    int j; // column index
    int x; // value
};
```

Meaning of each field

Field	Meaning
<code>i</code>	Row index
<code>j</code>	Column index
<code>x</code>	Value stored at that position

Example:

```
struct Element e;
e.i = 2;
```

```
e.j = 4;  
e.x = 9;
```

This means:

At row 2, column 4, the value is 9.

So the coordinate is:

(2, 4, 9)

7. The Sparse Structure

The Sparse structure stores information about the whole sparse matrix.

```
struct Sparse {  
    int m;           // number of rows  
    int n;           // number of columns  
    int num;         // number of non-zero elements  
    struct Element *ele; // dynamic array of non-zero elements  
};
```

Meaning of each field

Field	Meaning
m	Total number of rows
n	Total number of columns
num	Number of non-zero elements
ele	Pointer to the dynamic array of stored non-zero elements

Important:

num does not mean $m * n$.

$m * n$ means the total number of positions in the full matrix.

num means only the number of non-zero values.

Example:

```
m = 5  
n = 5  
num = 5
```

This means:

```
The full matrix has 25 positions.  
Only 5 values are non-zero.  
Only 5 values are stored.
```

8. Why We Use Dynamic Memory

In C, we use `malloc()` because we may not know the number of non-zero elements before the program runs.

The user may enter:

```
num = 3
```

or:

```
num = 50
```

or:

```
num = 1000
```

So we first read `num`, then allocate memory for exactly `num` `Element` records.

For one matrix:

```
s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));
```

For the result matrix:

```
sum->ele = (struct Element *) malloc((s1->num + s2->num) * sizeof(struct Element));
```

9. Why the Result Array Size Is `s1->num + s2->num`

When adding two sparse matrices, we do not know the final number of non-zero values immediately.

The largest possible result happens when no coordinates overlap.

Example:

```
s1 has 3 non-zero elements.  
s2 has 4 non-zero elements.
```

If none of their positions are the same, the result needs:

```
3 + 4 = 7 elements
```

So we allocate:

```
s1->num + s2->num
```

But the actual result may be smaller.

If two values have the same row and column, they are added and stored as one result element.

Example:

```
s1 has (2, 5, 7)  
s2 has (2, 5, 6)
```

They overlap at `(2, 5)`.

So:

```
7 + 6 = 13
```

The result stores:

```
(2, 5, 13)
```

Two input elements become one output element.

That is why we must set:

```
sum->num = k;
```

not:

```
sum->num = s1->num + s2->num;
```

10. Important Assumption: Row-Major Order

The sparse elements should be stored in row-major order.

Row-major order means:

1. Sort by row first.
2. If rows are equal, sort by column.

Good order:

```
(0, 0, 5)
(0, 3, 8)
(1, 2, 7)
(3, 1, 4)
```

Bad order:

```
(3, 1, 4)
(0, 0, 5)
(1, 2, 7)
(0, 3, 8)
```

The addition algorithm moves forward only. It does not go back and search again.

So if the elements are not sorted properly, the result may be wrong.

11. The `create()` Function

The `create()` function reads a sparse matrix from the user.

It reads:

1. Matrix dimensions: `m` and `n`
2. Number of non-zero elements: `num`
3. Each non-zero element: row, column, value

Code

```
void create(struct Sparse *s) {
    int i;

    printf("Enter matrix dimensions: ");
    scanf("%d %d", &s->m, &s->n);

    printf("Enter number of non-zero elements: ");
    scanf("%d", &s->num);

    s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));

    if (s->ele == NULL && s->num > 0) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    printf("Enter row, column, and value for each non-zero element:\n");

    for (i = 0; i < s->num; i++) {
        scanf("%d %d %d", &s->ele[i].i, &s->ele[i].j, &s->ele[i].x);
    }
}
```

12. Why `create()` Receives a Pointer

The function header is:

```
void create(struct Sparse *s)
```

Here, `s` is a pointer to a `Sparse` structure.

We pass a pointer because `create()` must modify the original matrix in `main()`.

In `main()`, we call:

```
create(&s1);  
create(&s2);
```

The `&` symbol means address.

So:

```
&s1
```

means:

```
address of s1
```

Because `create()` receives the address, it can fill the original structure.

13. Why We Use `->` Inside `create()`

Inside `create()`, `s` is a pointer.

So we use the arrow operator:

```
s->m  
s->n  
s->num  
s->ele
```

Do not use:

```
s.m  
s.n  
s.num  
s.ele
```

Simple rule:

Situation	Operator
You have a normal structure variable	.
You have a pointer to a structure	->

14. The `display()` Function

The `display()` function prints the sparse matrix as a full matrix.

Even though the program stores only non-zero values, `display()` prints all positions.

If a position is stored, it prints the stored value.

If a position is not stored, it prints `0`.

Code

```
void display(struct Sparse s) {
    int i, j, k = 0;

    for (i = 0; i < s.m; i++) {
        for (j = 0; j < s.n; j++) {
            if (k < s.num && i == s.ele[k].i && j == s.ele[k].j) {
                printf("%d ", s.ele[k].x);
                k++;
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}
```

15. Why `display()` Uses `.` Instead of `->`

The function header is:

```
void display(struct Sparse s)
```

Here, `s` is not a pointer.

It is a normal structure variable.

So we use dot:

```
s.m  
s.n  
s.num  
s.ele
```

not arrow:

```
s->m
```

16. What `k` Does in `display()`

In `display()`:

```
int i, j, k = 0;
```

`k` tracks the current non-zero element in the stored array.

Think of `k` as a finger pointing to the next stored non-zero value.

The loops move through every position in the full matrix.

For each position `(i, j)`, the program checks:

```
if (k < s.num && i == s.ele[k].i && j == s.ele[k].j)
```

This means:

1. Is `k` still inside the valid range?
2. Is the current row equal to the stored row?
3. Is the current column equal to the stored column?

If yes, print the stored value and move `k` forward.

If no, print `0`.

17. The `add()` Function

The `add()` function is the main function of Lesson 6.

It adds two sparse matrices and returns a pointer to the result.

Function header

```
struct Sparse *add(struct Sparse *s1, struct Sparse *s2)
```

This means:

Part	Meaning
<code>struct Sparse *</code> before <code>add</code>	The function returns a pointer to a <code>Sparse</code> structure.
<code>add</code>	Function name.
<code>struct Sparse *s1</code>	First sparse matrix pointer.
<code>struct Sparse *s2</code>	Second sparse matrix pointer.

18. Why `add()` Returns a Pointer

The result matrix is created inside the `add()` function.

If we created it as a normal local variable like this:

```
struct Sparse sum;  
return &sum;
```

that would be wrong.

Why?

Because `sum` would disappear when the function ends.

So we must create it on the heap:

```
sum = (struct Sparse *) malloc(sizeof(struct Sparse));
```

Heap memory remains available after the function returns.

So it is safe to return:

```
return sum;
```

19. Step-by-Step Logic of `add()`

The `add()` function does these steps:

1. Check if matrix dimensions match.
2. Allocate memory for the result structure.
3. Copy the matrix dimensions into the result.
4. Allocate memory for the result element array.
5. Set `i = 0`, `j = 0`, `k = 0`.
6. Compare elements from `s1` and `s2`.
7. Copy the smaller coordinate into the result.
8. If coordinates match, add the values.
9. Copy leftovers from `s1` or `s2`.
10. Set `sum->num = k`.
11. Return `sum`.

20. Full `add()` Function Code

```
struct Sparse *add(struct Sparse *s1, struct Sparse *s2) {
    struct Sparse *sum;
    int i, j, k;

    if (s1->m != s2->m || s1->n != s2->n) {
        return NULL;
    }

    sum = (struct Sparse *) malloc(sizeof(struct Sparse));

    if (sum == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    sum->m = s1->m;
    sum->n = s1->n;

    sum->ele = (struct Element *) malloc((s1->num + s2->num) * sizeof(struct
    Element));
```

```

if (sum->ele == NULL && (s1->num + s2->num) > 0) {
    printf("Memory allocation failed.\n");
    free(sum);
    exit(1);
}

i = 0;
j = 0;
k = 0;

while (i < s1->num && j < s2->num) {
    if (s1->ele[i].i < s2->ele[j].i) {
        sum->ele[k++] = s1->ele[i++];
    }
    else if (s1->ele[i].i > s2->ele[j].i) {
        sum->ele[k++] = s2->ele[j++];
    }
    else {
        if (s1->ele[i].j < s2->ele[j].j) {
            sum->ele[k++] = s1->ele[i++];
        }
        else if (s1->ele[i].j > s2->ele[j].j) {
            sum->ele[k++] = s2->ele[j++];
        }
        else {
            sum->ele[k].i = s1->ele[i].i;
            sum->ele[k].j = s1->ele[i].j;
            sum->ele[k].x = s1->ele[i].x + s2->ele[j].x;

            i++;
            j++;
            k++;
        }
    }
}

while (i < s1->num) {
    sum->ele[k++] = s1->ele[i++];
}

while (j < s2->num) {
    sum->ele[k++] = s2->ele[j++];
}

sum->num = k;

```

```
    return sum;
}
```

21. Dimension Check

Before adding, the program checks:

```
if (s1->m != s2->m || s1->n != s2->n) {
    return NULL;
}
```

Two matrices can be added only if they have the same number of rows and columns.

Example:

```
3 x 4 + 3 x 4 is allowed.
3 x 4 + 4 x 3 is not allowed.
```

Why?

Because matrix addition happens position by position.

Each position in the first matrix must match a position in the second matrix.

22. Allocating the Result Matrix

The result matrix is allocated using:

```
sum = (struct Sparse *) malloc(sizeof(struct Sparse));
```

This creates memory for one `Sparse` structure.

Then we copy the dimensions:

```
sum->m = s1->m;
sum->n = s1->n;
```

We can copy from `s1` because the dimension check already proved that `s1` and `s2` have the same size.

23. Allocating the Result Element Array

The result array is allocated using:

```
sum->ele = (struct Element *) malloc((s1->num + s2->num) * sizeof(struct Element));
```

This creates enough space for the worst case.

Worst case:

```
No coordinates overlap.
```

So all non-zero values from both matrices must be stored in the result.

24. The Three Pointers: `i`, `j`, and `k`

The addition algorithm uses three index variables:

```
i = 0;  
j = 0;  
k = 0;
```

Variable	Meaning
<code>i</code>	Current element in <code>s1</code>
<code>j</code>	Current element in <code>s2</code>
<code>k</code>	Current writing position in <code>sum</code>

Beginner mental model:

- `i` points to the current element in the first matrix.
 - `j` points to the current element in the second matrix.
 - `k` points to where the next result element should be written.
-

25. Main Loop Condition

The main loop is:


```
while (i < s1->num && j < s2->num)
```

This means:

Keep comparing while both matrices still have elements left.

The loop stops when either `s1` or `s2` finishes.

After the loop, we copy leftovers from the matrix that still has elements.

26. Merge Rule 1: `s1` Row Is Smaller

Code:

```
if (s1->ele[i].i < s2->ele[j].i) {  
    sum->ele[k++] = s1->ele[i++];  
}
```

Meaning:

The current element in `s1` comes before the current element in `s2`.

So copy the `s1` element into the result.

Then move:

```
i forward  
k forward
```

Example:

```
s1 = (1, 3, 7)  
s2 = (2, 1, 5)
```

Row comparison:

```
1 < 2
```

So copy:

(1, 3, 7)

27. Merge Rule 2: s2 Row Is Smaller

Code:

```
else if (s1->ele[i].i > s2->ele[j].i) {  
    sum->ele[k++] = s2->ele[j++];  
}
```

Meaning:

The current element in s2 comes before the current element in s1.

So copy the s2 element into the result.

Then move:

j forward
k forward

Example:

s1 = (3, 0, 4)
s2 = (1, 2, 8)

Row comparison:

3 > 1

So copy:

(1, 2, 8)

28. Merge Rule 3: Rows Equal, **s1** Column Is Smaller

If rows are equal, compare columns.

Code:

```
if (s1->ele[i].j < s2->ele[j].j) {  
    sum->ele[k++] = s1->ele[i++];  
}
```

Example:

```
s1 = (2, 1, 6)  
s2 = (2, 4, 9)
```

Rows are equal:

```
2 == 2
```

Compare columns:

```
1 < 4
```

So copy **s1**:

```
(2, 1, 6)
```

29. Merge Rule 4: Rows Equal, **s2** Column Is Smaller

Code:

```
else if (s1->ele[i].j > s2->ele[j].j) {  
    sum->ele[k++] = s2->ele[j++];  
}
```

Example:

```
s1 = (2, 5, 6)
s2 = (2, 3, 9)
```

Rows are equal:

```
2 == 2
```

Compare columns:

```
5 > 3
```

So copy `s2`:

```
(2, 3, 9)
```

30. Merge Rule 5: Same Row and Same Column

This is the real addition case.

Code:

```
else {
    sum->ele[k].i = s1->ele[i].i;
    sum->ele[k].j = s1->ele[i].j;
    sum->ele[k].x = s1->ele[i].x + s2->ele[j].x;

    i++;
    j++;
    k++;
}
```

Example:

```
s1 = (2, 5, 7)
s2 = (2, 5, 6)
```

Same row:

```
2 == 2
```

Same column:

```
5 == 5
```

Add the values:

```
7 + 6 = 13
```

Store:

```
(2, 5, 13)
```

Then move all three pointers:

```
i++  
j++  
k++
```

Why move both `i` and `j`?

Because one element from `s1` and one element from `s2` were both used.

Why move `k`?

Because one result element was written.

31. Copying Leftovers

The main loop runs only while both matrices still have elements.

```
while (i < s1->num && j < s2->num)
```

If one matrix finishes first, the other matrix may still have remaining elements.

These remaining elements are called leftovers.

They must be copied into the result.

Copy leftovers from `s1`

```
while (i < s1->num) {  
    sum->ele[k++] = s1->ele[i++];  
}
```

Copy leftovers from `s2`

```
while (j < s2->num) {  
    sum->ele[k++] = s2->ele[j++];  
}
```

Why copy leftovers?

Because if a value exists only in one matrix, it remains unchanged in the sum.

32. Why `sum->num = k`

At the start, we allocate maximum possible space:

```
s1->num + s2->num
```

But the actual number of stored result elements may be smaller.

The variable `k` counts how many result elements were actually written.

So we must write:

```
sum->num = k;
```

Wrong:

```
sum->num = s1->num + s2->num;
```

Why wrong?

Because if some coordinates overlap, the actual result count is smaller.

33. The `main()` Function

The `main()` function connects everything.

It creates two sparse matrices, adds them, displays them, and frees memory.

Code

```
int main() {
    struct Sparse s1, s2;
    struct Sparse *s3;

    printf("Enter first sparse matrix:\n");
    create(&s1);

    printf("\nEnter second sparse matrix:\n");
    create(&s2);

    s3 = add(&s1, &s2);

    if (s3 == NULL) {
        printf("\nMatrices cannot be added because their dimensions are
different.\n");

        free(s1.ele);
        free(s2.ele);

        return 0;
    }

    printf("\nFirst Matrix:\n");
    display(s1);

    printf("\nSecond Matrix:\n");
    display(s2);

    printf("\nSum Matrix:\n");
    display(*s3);

    free(s1.ele);
    free(s2.ele);
    free(s3->ele);
    free(s3);
}
```

```
    return 0;  
}
```

34. Why We Write `display(*s3)`

In `main()`, we declare:

```
struct Sparse *s3;
```

This means `s3` is a pointer.

But `display()` expects a normal structure value:

```
void display(struct Sparse s)
```

So this is wrong:

```
display(s3);
```

Because `s3` is a pointer.

Correct:

```
display(*s3);
```

The `*` means:

```
Go to the Sparse object that s3 points to.
```

So `*s3` is the actual result matrix.

35. Why We Use `free()`

Memory created with `malloc()` should be released using `free()`.

In this program, memory is allocated for:

1. `s1.ele`
2. `s2.ele`
3. `s3`
4. `s3->ele`

So we free them:

```
free(s1.ele);
free(s2.ele);
free(s3->ele);
free(s3);
```

Important:

Free the element array inside `s3` first:

```
free(s3->ele);
```

Then free the structure itself:

```
free(s3);
```

If you free `s3` first, you may lose access to `s3->ele`.

36. Complete C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Element {
    int i; // row index
    int j; // column index
    int x; // value
};

struct Sparse {
    int m;           // rows
    int n;           // columns
    int num;         // number of non-zero elements
```

```

    struct Element *ele; // dynamic array
};

void create(struct Sparse *s) {
    int i;

    printf("Enter matrix dimensions: ");
    scanf("%d %d", &s->m, &s->n);

    printf("Enter number of non-zero elements: ");
    scanf("%d", &s->num);

    s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));

    if (s->ele == NULL && s->num > 0) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    printf("Enter row, column, and value for each non-zero element:\n");

    for (i = 0; i < s->num; i++) {
        scanf("%d %d %d", &s->ele[i].i, &s->ele[i].j, &s->ele[i].x);
    }
}

void display(struct Sparse s) {
    int i, j, k = 0;

    for (i = 0; i < s.m; i++) {
        for (j = 0; j < s.n; j++) {
            if (k < s.num && i == s.ele[k].i && j == s.ele[k].j) {
                printf("%d ", s.ele[k].x);
                k++;
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

struct Sparse *add(struct Sparse *s1, struct Sparse *s2) {
    struct Sparse *sum;
    int i, j, k;

    if (s1->m != s2->m || s1->n != s2->n) {
        return NULL;
    }
}

```

```

}

sum = (struct Sparse *) malloc(sizeof(struct Sparse));

if (sum == NULL) {
    printf("Memory allocation failed.\n");
    exit(1);
}

sum->m = s1->m;
sum->n = s1->n;

sum->ele = (struct Element *) malloc((s1->num + s2->num) * sizeof(struct
Element));

if (sum->ele == NULL && (s1->num + s2->num) > 0) {
    printf("Memory allocation failed.\n");
    free(sum);
    exit(1);
}

i = 0;
j = 0;
k = 0;

while (i < s1->num && j < s2->num) {
    if (s1->ele[i].i < s2->ele[j].i) {
        sum->ele[k++] = s1->ele[i++];
    }
    else if (s1->ele[i].i > s2->ele[j].i) {
        sum->ele[k++] = s2->ele[j++];
    }
    else {
        if (s1->ele[i].j < s2->ele[j].j) {
            sum->ele[k++] = s1->ele[i++];
        }
        else if (s1->ele[i].j > s2->ele[j].j) {
            sum->ele[k++] = s2->ele[j++];
        }
        else {
            sum->ele[k].i = s1->ele[i].i;
            sum->ele[k].j = s1->ele[i].j;
            sum->ele[k].x = s1->ele[i].x + s2->ele[j].x;

            i++;
            j++;
            k++;
        }
    }
}

```

```

    }
}

while (i < s1->num) {
    sum->ele[k++] = s1->ele[i++];
}

while (j < s2->num) {
    sum->ele[k++] = s2->ele[j++];
}

sum->num = k;

return sum;
}

int main() {
    struct Sparse s1, s2;
    struct Sparse *s3;

    printf("Enter first sparse matrix:\n");
    create(&s1);

    printf("\nEnter second sparse matrix:\n");
    create(&s2);

    s3 = add(&s1, &s2);

    if (s3 == NULL) {
        printf("\nMatrices cannot be added because their dimensions are
different.\n");

        free(s1.ele);
        free(s2.ele);

        return 0;
    }

    printf("\nFirst Matrix:\n");
    display(s1);

    printf("\nSecond Matrix:\n");
    display(s2);

    printf("\nSum Matrix:\n");
    display(*s3);

    free(s1.ele);

```

```
    free(s2.ele);
    free(s3->ele);
    free(s3);

    return 0;
}
```

37. Test Case 1 — Partial Overlap

This test adds:

1. A diagonal matrix of 1 s
2. A first-column matrix of 2 s

Only one position overlaps: (0, 0).

38. Test Case 1 Input for Matrix 1

Matrix 1 is a diagonal matrix of 1 s.

Input:

```
5 5
5
0 0 1
1 1 1
2 2 1
3 3 1
4 4 1
```

Meaning:

```
m = 5
n = 5
num = 5
```

Stored elements:

```
(0, 0, 1)
(1, 1, 1)
```

```
(2, 2, 1)
(3, 3, 1)
(4, 4, 1)
```

Full matrix view:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

39. Test Case 1 Input for Matrix 2

Matrix 2 is a first-column matrix of 2 s.

Input:

```
5 5
5
0 0 2
1 0 2
2 0 2
3 0 2
4 0 2
```

Stored elements:

```
(0, 0, 2)
(1, 0, 2)
(2, 0, 2)
(3, 0, 2)
(4, 0, 2)
```

Full matrix view:

```
2 0 0 0 0
2 0 0 0 0
2 0 0 0 0
```

```
2 0 0 0 0
2 0 0 0 0
```

40. Test Case 1 Addition Result

Only position (0, 0) overlaps.

At (0, 0) :

```
1 + 2 = 3
```

All other non-zero values are copied.

Result:

```
3 0 0 0 0
2 1 0 0 0
2 0 1 0 0
2 0 0 1 0
2 0 0 0 1
```

Result Coordinate List:

```
(0, 0, 3)
(1, 0, 2)
(1, 1, 1)
(2, 0, 2)
(2, 2, 1)
(3, 0, 2)
(3, 3, 1)
(4, 0, 2)
(4, 4, 1)
```

So:

```
sum->num = 9
```

Why 9?

Matrix 1 has 5 non-zero elements.

Matrix 2 has 5 non-zero elements.

Maximum possible result size:

$$5 + 5 = 10$$

But one coordinate overlaps: $(0, 0)$.

So two elements become one.

Final result count:

$$10 - 1 = 9$$

41. Test Case 2 — Full Overlap

This test adds:

1. A diagonal matrix of 1 s
2. A diagonal matrix of 5 s

Every coordinate overlaps.

42. Test Case 2 Matrix 1

Matrix 1:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

Coordinate List:

```
(0, 0, 1)
(1, 1, 1)
(2, 2, 1)
```


(3, 3, 1)
(4, 4, 1)

43. Test Case 2 Matrix 2

Matrix 2:

```
5 0 0 0 0
0 5 0 0 0
0 0 5 0 0
0 0 0 5 0
0 0 0 0 5
```

Coordinate List:

```
(0, 0, 5)
(1, 1, 5)
(2, 2, 5)
(3, 3, 5)
(4, 4, 5)
```

44. Test Case 2 Addition Result

Every coordinate overlaps.

At every diagonal position:

1 + 5 = 6

Result:

```
6 0 0 0 0
0 6 0 0 0
0 0 6 0 0
0 0 0 6 0
0 0 0 0 6
```

Result Coordinate List:

```
(0, 0, 6)
(1, 1, 6)
(2, 2, 6)
(3, 3, 6)
(4, 4, 6)
```

So:

```
sum->num = 5
```

Why 5?

Matrix 1 has 5 non-zero elements.

Matrix 2 has 5 non-zero elements.

Maximum possible result size:

```
5 + 5 = 10
```

But every coordinate overlaps.

So every pair becomes one result element.

Final result count:

```
5
```

45. Dry Run of One Important Overlap

Suppose:

```
s1->ele[i] = (0, 0, 1)
s2->ele[j] = (0, 0, 2)
```

Initial values:

```
i = 0  
j = 0  
k = 0
```

Compare rows:

```
0 == 0
```

Rows are equal, so compare columns:

```
0 == 0
```

Same row and same column.

Add values:

```
1 + 2 = 3
```

Store:

```
sum->ele[0] = (0, 0, 3)
```

Move pointers:

```
i = 1  
j = 1  
k = 1
```

This is the overlap case.

46. Dry Run of a Non-Overlap Copy

Suppose:

```
s1->ele[i] = (1, 1, 1)  
s2->ele[j] = (1, 0, 2)
```

Compare rows:

```
1 == 1
```

Rows are equal, so compare columns:

```
1 > 0
```

So `s2` comes first.

Copy:

```
sum->ele[k] = (1, 0, 2)
```

Move:

```
j++  
k++
```

`i` does not move because the `s1` element was not used yet.

47. Complexity Analysis

Sparse matrix addition does not visit every matrix position.

It only scans the stored non-zero elements.

If:

```
s1->num = number of non-zero elements in first matrix  
s2->num = number of non-zero elements in second matrix
```

Then:

```
Time complexity = O(s1->num + s2->num)
```

Why?

Because each non-zero element is visited at most once.

The pointer `i` only moves forward.

The pointer `j` only moves forward.

No element is repeatedly searched.

48. Space Complexity

The result array is allocated with maximum possible size:

```
s1->num + s2->num
```

So the worst-case result space is:

```
O(s1->num + s2->num)
```

Why?

Because if there are no overlapping coordinates, all non-zero elements from both matrices must be stored in the result.

49. Complexity Table

Operation	Time complexity	Space complexity	Reason
<code>create()</code>	<code>O(num)</code>	<code>O(num)</code>	Reads and stores each non-zero element once.
<code>display()</code>	<code>O(m * n)</code>	<code>O(1)</code> extra	Visits every visible matrix position.
<code>add()</code>	<code>O(s1->num + s2->num)</code>	<code>O(s1->num + s2->num)</code>	Scans both sparse lists once and creates result storage.
<code>free()</code>	<code>O(1)</code> per pointer	<code>O(1)</code>	Releases allocated memory.

50. Why Sparse Addition Is Efficient

Normal matrix addition checks every position.

If a matrix has:

m rows and n columns

then normal addition checks:

m * n positions

So normal matrix addition takes:

$O(m * n)$

Sparse matrix addition only checks stored non-zero elements.

So if the matrix is very sparse, this is much faster.

Example:

Full matrix size = 1000 x 1000 = 1,000,000 positions
s1 non-zero elements = 100
s2 non-zero elements = 150

Normal addition checks about:

1,000,000 positions

Sparse addition checks about:

100 + 150 = 250 stored elements

That is the advantage of sparse matrix addition.

51. Common Beginner Mistakes

Mistake	Why it is wrong	Correct idea
Using <code>new</code> in the C version	<code>new</code> is C++ style.	Use <code>malloc</code> .

Mistake	Why it is wrong	Correct idea
Forgetting the dimension check	Matrices with different sizes cannot be added.	Check <code>m</code> and <code>n</code> first.
Allocating only <code>max(s1->num, s2->num)</code>	Not enough when no coordinates overlap.	Allocate <code>s1->num + s2->num</code> .
Setting <code>sum->num = s1->num + s2->num</code>	That is only the maximum capacity.	Set <code>sum->num = k</code> .
Forgetting to copy leftovers	Values remaining in one matrix would be lost.	Copy leftover elements after the main loop.
Moving only one pointer when coordinates match	Both input elements were used.	Move <code>i</code> , <code>j</code> , and <code>k</code> .
Comparing columns before rows	The list is sorted row-first.	Compare rows first, then columns.
Calling <code>display(s3)</code>	<code>s3</code> is a pointer, but <code>display()</code> expects a value.	Use <code>display(*s3)</code> .
Using <code>.</code> with a pointer	Dot is for normal variables.	Use <code>-></code> with pointers.
Using <code>-></code> with a normal structure	Arrow is for pointers.	Use <code>.</code> with normal structures.
Forgetting <code>free()</code>	Causes memory leaks.	Free all allocated memory.
Entering elements randomly	Merge algorithm assumes sorted input.	Enter elements in row-major order.

52. Important Code Rules

Rule 1: Use `->` for pointers

```
s1->m
s1->n
s1->num
s1->ele
```

Rule 2: Use `.` for normal structures

```
s1.m
s1.n
```

```
s1.num  
s1.ele
```

Rule 3: Allocate after reading `num`

```
s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));
```

Rule 4: Result capacity should be maximum possible size

```
sum->ele = (struct Element *) malloc((s1->num + s2->num) * sizeof(struct  
Element));
```

Rule 5: Final result count is `k`

```
sum->num = k;
```

Rule 6: Dereference result pointer when displaying

```
display(*s3);
```

Rule 7: Free memory after use

```
free(s1.ele);  
free(s2.ele);  
free(s3->ele);  
free(s3);
```

53. Beginner Mental Model

Think of each sparse matrix as a sorted list of important locations.

Each location has:

```
row, column, value
```

The addition function compares the first unused location from each list.

- If one location comes earlier, copy it.

- If both locations are the same, add their values.
- Move forward through the lists.
- When one list ends, copy the rest of the other list.

This is just like merging two sorted lists.

54. One-Line Memory Trick

Sparse matrix addition in C uses `malloc` to create the result, merges two sorted Coordinate Lists using `i`, `j`, and `k`, and sets `sum->num = k` because `k` is the actual number of result elements.

55. Quick Self-Check Questions

Question 1

What does `num` mean?

Answer

`num` means the number of non-zero elements.

It does not mean `m * n`.

Question 2

When can two sparse matrices be added?

Answer

They can be added only when they have the same number of rows and columns.

```
s1->m == s2->m  
s1->n == s2->n
```

Question 3

Why do we allocate result space as `s1->num + s2->num`?

Answer

Because in the worst case, no coordinates overlap, so the result may need to store all non-zero elements from both matrices.

Question 4

What does `i` do?

Answer

`i` points to the current element in the first sparse matrix.

Question 5

What does `j` do?

Answer

`j` points to the current element in the second sparse matrix.

Question 6

What does `k` do?

Answer

`k` points to the current writing position in the result matrix.

It also counts the actual number of result elements.

Question 7

Why do we write `sum->num = k`?

Answer

Because `k` is the actual number of non-zero elements written into the result.

Question 8

Why do we write `display(*s3)`?

Answer

Because `s3` is a pointer, but `display()` expects a normal structure value.

`*s3` gives the actual structure that `s3` points to.

Question 9

Why do we copy leftovers?

Answer

Because when one sparse list finishes first, the other list may still have non-zero elements that must appear in the result.

Question 10

What is the time complexity of sparse matrix addition?

Answer

$O(s1 \rightarrow \text{num} + s2 \rightarrow \text{num})$

Question 11

What is the space complexity of sparse matrix addition?

Answer

$O(s1 \rightarrow \text{num} + s2 \rightarrow \text{num})$

Question 12

Why should input elements be in row-major order?

Answer

Because the merge algorithm moves forward only. It assumes that elements are sorted by row first and column second.

56. Lesson 6 Final Summary

Lesson 6 completes the sparse matrix addition program in C.

The program uses:

```
malloc
```

for dynamic memory allocation.

It uses:

```
i, j, k
```

for merge-style sparse matrix addition.

It uses:

```
sum->num = k
```

to store the actual number of result elements.

It uses:

```
display(*s3)
```

because the result matrix is returned as a pointer.

It uses:

```
free()
```

to release memory after use.

The most important idea is:

Sparse matrix addition does not add every cell.
It only merges the stored non-zero elements.

That is why sparse matrix addition is efficient.

After Lesson 6, you have completed the procedural sparse matrix addition part. The next step is Lesson 7, where the sparse matrix program is converted into a C++ class version.